LEARN-AI · RESEARCH TRADING PLATFORM

Broker & Live Trading

A plain-English user manual: architecture, the pre-trade checklist, deploying and adding strategies

Audience: the operator (you) — no deep finance background assumed.
Scope: the live-trading subsystem (Interactive Brokers *paper* account).
Version: 1.0 · Generated 2026-06-01 · Source of truth: this repository.

What's in this manual

> **How to read this.** Sections 1–4 explain *what the system is*. Sections 5–9 are the day-to-day operator guide — this is where your question "how do I keep a clean pre-trade checklist so I can deploy" is answered. Section 10 is for when you (or a developer) want to *add a new strategy*.

1 · The 60-second overview

This platform lets you take a **trading strategy** — a set of rules like "buy SPY when these indicators line up, sell when they don't" — and run it **automatically** against a brokerage account. The piece that does this is what we loosely call *"the broker"*: the live-trading subsystem.

It does three jobs:

1. **Deploy** — freeze an exact copy of your strategy + code into a tamper-evident record so you know precisely what is running.
2. **Run** — start a process that connects to **Interactive Brokers (IBKR)**, watches live price bars, and places orders when your rules fire.
3. **Supervise** — show you, at a glance, whether the bot is healthy and allowed to trade, and let you pause, resume, stop, or flatten it.

> **The one idea to take away:** nothing trades unless a stack of safety checks is green. Those checks are the "pre-trade checklist." Most of this manual is about understanding those checks and keeping them green so a deploy goes through cleanly.

Where you click

Everything operator-facing lives under the **Broker** area of the web app (`http://localhost:4200`):

Page	URL path	What it's for
Instances console	`/broker/instances`	Your home base — see every bot, its health, start/stop/pause.
Deploy form	`/broker/deploy`	Create a new live run from a strategy (Stage 1 of deploy).
Account monitor / orders / reconciliation	`/broker/...`	Account positions, order history, and trade reconciliation.

2 · Words you need

These terms appear everywhere. Learn these eight and the rest of the manual reads easily.

Strategy
The trading rules themselves (which indicators, when to buy, when to sell). Lives as code or as a declarative *spec* file. Example: "SPY EMA(5)/EMA(10) crossover with an RSI gate."

Indicator
A number computed from price history — e.g. a moving average (EMA), RSI, MACD, ADX. Strategies combine indicators into buy/sell conditions. Each indicator in this repo is a faithful port of a reference implementation, proven equal to within a tiny tolerance.

Strategy *instance*
One configured, named bot — e.g. "SPY EMA Crossover #1." It has a stable ID (`strategy_instance_id`) that survives restarts and crashes. **This is the thing you manage.**

Run
One execution (one process lifetime) of an instance. If the bot restarts, it's a new run of the *same* instance. A run has a `run_id` .

Ledger
The file (`run_ledger.json`) that records exactly what a run is: which code commit, which strategy, which account, which backtest it's anchored to. It is the bot's "identity card."

Readiness
A live verdict — READY BLOCKED DEGRADED UNKNOWN — answering "is this bot allowed to act on the next price bar?" Computed by the engine itself and shown verbatim in the UI.

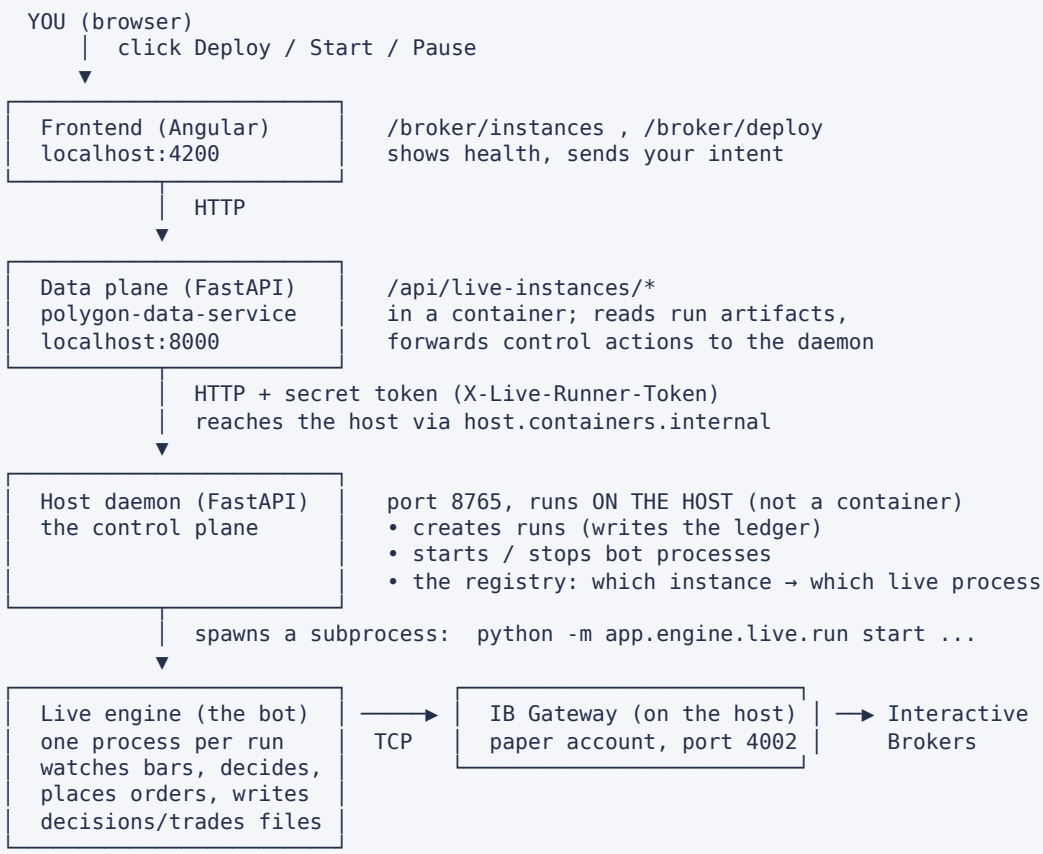
The daemon
A small always-on program running on your *host machine* (the computer where IBKR's Gateway runs), not inside the containers. It is the only thing allowed to start/stop bot processes and create runs.

QC anchor
A QuantConnect (QC) cloud backtest that every live run must point to, plus a committed copy of the algorithm that produced it. It is your independent proof that the strategy was validated. No anchor → no deploy.

> **Instance vs. run, in one line:** the *instance* is the bot you name and manage; a *run* is one time it was switched on. The console is organized around instances; runs come and go underneath.

3 · How it's architected — the moving parts

The system is three layers. The unusual part — and the key to understanding it — is the **host daemon**, which sits *outside* the containers because the broker forces it to.



Why the daemon lives on the host

Interactive Brokers ties an API session to the same machine/IP as the login. A containerized process has a different network identity, so it cannot safely own the IBKR session. The daemon therefore runs natively on the host, right next to IB Gateway, and is the single authority over bot processes. Two consequences you'll feel:

- The daemon and IB Gateway are **not** started by `./restart.sh` or `podman compose up`. They are separate host programs you start yourself. If the console says "Live engine unavailable," the daemon isn't running.
- The container talks to the daemon over the network, so the daemon requires a **shared secret** (auto-generated token) on every request — this is what lets it bind beyond loopback safely (ADR 0007).

What each layer is responsible for

Layer	Owns	Does NOT do
Frontend (Angular)	Showing health/readiness, capturing your intent, friendly error messages.	Any trading logic or git/process work.
Data plane (FastAPI, container)	Reading run artifacts from disk; relaying control actions to the daemon; merging status.	Starting processes or touching the broker directly.

Host daemon (FastAPI, host)	Creating runs (git check + ledger), starting/stopping processes, the instance→process registry.	Running the strategy itself.
Live engine (subprocess)	Connecting to IBKR, consolidating bars, evaluating the strategy, placing orders, emitting readiness, writing receipts.	Deciding whether it is *allowed* to exist (that's the daemon).

> **Design records.** The decisions above are written up as ADRs in `docs/architecture/adrs/` : 0003 (host topology), 0004 (instance-addressed console), 0005 (engine-authored readiness), 0006 (deploy control plane), 0007 (daemon shared-secret auth). Read those if you want the "why" behind any rule in this manual.

4 · Safety: why this only trades a paper account

This subsystem is built and shipped for **paper trading** — a simulated IBKR account that uses real market data but fake money. Real-money trading is deliberately out of scope. Several independent guards enforce this, so a misconfiguration fails closed rather than placing a real order:

Guard	What it enforces
Read-only default	The deploy form defaults to **paper**; the engine's `readonly` mode refuses to submit orders. A "live" switch exists but is gated behind an explicit confirmation step and is not the supported path.
Mode lock	The broker mode defaults to `paper` and refuses to silently drift to `live`.
Port vs. mode check	Paper mode rejects live ports (and vice-versa), so you can't accidentally point at a live Gateway.
Account sentinel	After connecting, the engine asserts the account ID starts with `DU` (IBKR's paper prefix). A mismatch disconnects immediately.
Daily order cap	A hard `max_orders_per_day` limit (default 4) blocks runaway trading.
Single symbol	Currently scoped to one symbol (SPY). A position in any other symbol trips a safety check.

> **Treat the "live" option as off-limits unless you have separately validated real-money infrastructure.** As the repository states: the backtesting engine is a research tool; live trading requires separately validated infrastructure. Stay on paper.

5 · The two checklists that gate a deploy

Your question — "how can I always have a clean pre-trade checklist so I can deploy" — has a precise answer, because there are actually **two** checklists at **two different moments**. Knowing which is which is half the battle.

	Checklist A — Deploy readiness	Checklist B — Morning pre-flight
When	On the Deploy form, *before* you create a run.	Each trading morning, *before* the bot's first decision.
Where you see it	`/broker/deploy` — "Now checks" and "Deploy checks" panels.	The instance console readiness panel; CLI `run pre-flight`.
Question it answers	"Can I create and (optionally) start this run right now?"	"Is it safe for this already-deployed bot to trade *today*?"
If it fails	Deploy is blocked; nothing is created.	The bot sits out the day; yesterday's record stays valid.

Checklist A — Deploy readiness (the Deploy form)

The form shows two small status panels. The first is checked continuously while you fill the form; the second is checked the instant you press **Deploy**.

"Now checks" (live, while you type)

Check	Green when…	If not green
Engine up	The host daemon answers.	Start the daemon on the host machine, then recheck.
Broker	IBKR Gateway is connected.	Log in to IB Gateway (paper) on the host.
Fields	All required fields are filled.	Complete the missing fields (see §7).
Fleet clear	No account-level state blocks new starts.	Either deploy-only (untick "start now") or clear the account state.

"Deploy checks" (verified at submit)

Check	Meaning	How to make it pass
Working tree clean	No uncommitted code changes in scope (`PythonDataService/`, `references/qc-shadow/`).	`git commit` or `git stash` your changes, then deploy again.
Spec matches strategy	The chosen spec file exists and matches the chosen strategy.	Pick the spec that corresponds to your strategy.

> **Why "working tree clean" matters.** The ledger stamps the exact git commit (`code_sha`) into the run's identity. If your working tree has uncommitted edits, the running code wouldn't match that commit — so the system refuses. This is what makes "what's deployed" provable.

Checklist B — Morning pre-flight (6 gates)

Once a bot is deployed, every trading morning it runs a 6-point pre-flight, like a pilot before takeoff. **All six must pass** or the bot refuses to place orders that session. These are detailed in §9; the full table with fixes is

there.

6 · How to always have a *clean* pre-trade checklist

This is the heart of your request. A "clean checklist" means: when you arrive at the Deploy form, every panel is green and the button works on the first try. The reliable way to get there is a **fixed routine**. Do these in order and a deploy almost never bounces.

The clean-deploy routine

1. **Commit everything first.** Run `git status`. If anything under `PythonDataService/` or `references/qc-shadow/` is modified, commit or stash it. This single habit clears the most common deploy failure (the 409 "dirty tree").
2. **Start the host pieces.** On the host machine: (a) launch IB Gateway and log in to the *paper* account, (b) start the host daemon. Confirm both show green on the connectivity strip before you touch the form.
3. **Have your QC anchor ready.** Before deploying you need two things from QuantConnect: the *backtest ID* and a committed copy of the algorithm under `references/qc-shadow/`. If the copy isn't committed, it won't appear in the picker and you can't deploy. (See §7 for what these are.)
4. **Fill every required field.** Strategy, spec, account ID, QC backtest ID, QC audit-copy, and an instance name. The "Fields" check goes green only when all are present.
5. **Decide deploy-only vs. start-now.** If "Fleet clear" is amber, untick "Start trading immediately" — you can create the run now and start it later from the console once the fleet is clear.
6. **Deploy.** If it bounces, the error tells you exactly which gate failed and how to fix it (§11). Fix that one thing and resubmit — the run ID is content-addressed, so re-submitting identical inputs is a safe no-op, not a duplicate.

> **Golden rule for a clean checklist:** *commit your code, bring up the host (Gateway + daemon), and have your QC anchor committed — before you open the form.* Those three habits keep both checklists green.

Keeping the morning pre-flight clean (so deployed bots actually trade)

A deploy can be clean yet the bot still sits out a morning if pre-flight fails. To keep pre-flight green:

- **Don't edit code on a deployed branch mid-week.** A dirty tree fails the clean-tree gate.
- **Keep the host clock synced.** The NTP gate needs the clock within ±1 second. Enable automatic time sync on the host OS.
- **Don't trade the bot's account by hand.** A surprise or short position trips the unexpected-position gate.
- **Never delete a run's files.** The ledger, yesterday's trade/reconciliation files, and the QC exports are all verified by hash each morning. Deleting or overwriting them blocks the next session.
- **If yesterday ended in a halt, deploy fresh.** A halt flag cannot be cleared by restarting the same run — read the halt reason, then create a new run.

7 · Deploying a strategy, step by step

Deploy is a three-stage pipeline (ADR 0006). You drive Stage 1 from the form; Stages 2–3 happen from the console.

Stage	What happens	Where
1. Create the run	Git clean-tree check, capture the commit, compute a content-addressed `run_id`, write the ledger.	Deploy form → daemon
2. Launch the process	Spawn the live engine subprocess that connects to IBKR.	"Start now" on the form, or Start in the console
3. Observe & control	Watch readiness; pause/resume/stop/flatten.	Instance console

The Deploy form fields, in plain English

Field	What to enter
Strategy	Pick from the engine's strategy list. This also auto-selects the matching spec.
Spec	The declarative strategy file (auto-filled from the strategy; can be set manually). E.g. `spy_ema_crossover.spec.json`.
Account ID	Your IBKR *paper* account, e.g. `DU1234567`. Pre-filled if the broker is connected; otherwise type it.
QC backtest ID	The QuantConnect cloud backtest identifier (the 32-hex Algorithm Id) that validates this strategy. Typed in by hand — there is no auto-lookup in v1.
QC audit-copy	Dropdown of committed files under `references/qc-shadow/` — the exact algorithm code that ran in that QC backtest. Only committed files appear.
Instance name	A stable ID for this bot, e.g. `spy-ema-prod-001`. Reuse it to manage the same bot across restarts.
Mode	Paper (default, recommended) or live (guarded). Keep Paper.
Hydrate policy	Whether to reuse saved indicator warmup state: `require` (default), `optional`, or `disabled`. `require` avoids re-paying the ~3h45m warmup each morning.
Max orders / day	Safety cap (default 4). Only used when starting.
Start trading immediately	If ticked, the run is launched right after it's created.

What the QC anchor is, and why it's mandatory

Every live run must point to a QuantConnect backtest *and* ship a committed copy of the algorithm that produced it. This gives you a three-way audit trail: *your engine's trades* ↔ *QuantConnect's trades* ↔ *the broker's fills*. If you don't yet have a QC backtest, you must run one first — the system will not let a strategy go live without this independent proof.

> **The "audit-copy picker" gotcha.** The dropdown lists only *git-committed* files in `references/qc-shadow/`. If you saved the copy but didn't commit it, it won't show up and the deploy field can't be filled.

> Commit it first. (Separately: on Linux/podman, an empty picker can also mean the container can't reach the loopback-only daemon — start/bind the daemon so the container can reach it.)

Deploying from the command line (equivalent of Stage 1 + 2)

The UI form is the recommended path, but the same thing is scriptable. Stage 1 (create the run):

```
cd PythonDataService
python -m app.engine.live.run init-ledger \
  --repo-root /home/inkant/Documents/learn-ai \
  --strategy-spec-path app/engine/strategy/spec/fixtures/spy_ema_crossover.spec.json \
  --qc-audit-copy-path references/qc-shadow/<your-copy>.py \
  --qc-cloud-backtest-id <32-hex-id> \
  --account-id DU1234567 \
  --start-date-ms <int64 ms UTC> \
  --strategy-instance-id spy-ema-prod-001
# → writes live_runs/<run_id>/run_ledger.json
```

Stage 2 (launch the engine):

```
python -m app.engine.live.run start --run-dir live_runs/<run_id>
```

Operator control (durable intent — survives a crash/reboot):

```
python -m app.engine.live.run pause   --strategy-instance-id spy-ema-prod-001
python -m app.engine.live.run resume  --strategy-instance-id spy-ema-prod-001
python -m app.engine.live.run stop    --strategy-instance-id spy-ema-prod-001
```

8 · Daily operation: start, pause, stop, monitor

Day-to-day you live in the **instance console** (`/broker/instances`). It is organized by *instance*, and for each one it merges three views into a single picture:

- **Process state** (from the daemon): is a process alive, what's its PID, which run is it bound to?
- **Readiness** (from the engine): the verdict and the per-gate detail.
- **Broker slice**: positions and pending orders attributed to this instance.

The readiness verdict, decoded

Verdict	Meaning	Your action
READY	All hard gates pass; the bot may act on the next bar.	Nothing — let it run.
BLOCKED	A hard gate failed (e.g. broker disconnected, order cap hit, emergency stop). The bot will not trade.	Open the failing gate; follow its fix hint.
DEGRADED	Hard gates pass but a soft gate warns (e.g. bars came from a fallback data source).	It can still trade; review the warning.
UNKNOWN	No authoritative readiness — usually a stopped/never-started instance.	Check process state; start it if intended.

Controls

Action	Effect
Start	Launch the engine for a deployed run.
Pause	Stay connected but stop placing new orders. Durable — remembered across restarts.
Resume	Return a paused bot to trading.
Stop	Shut the bot down. Restarting requires Start (or a fresh deploy).
Flatten	Force-close all positions immediately (emergency).
Reconcile	Verify the broker's account state matches what the engine expects.

> **"Desired state" is sticky.** When you Pause/Stop, that intent is written to disk (`desired_state.json`). If the bot or machine restarts, it comes back in the state you left it — a paused bot boots paused; a stopped bot refuses to auto-start. You decide when it trades, not the accident of a reboot.

9 · The morning pre-flight gate (6 checks)

This is Checklist B in detail. It runs once at session start. A failure here is *not* a disaster — it just skips today; yesterday's record stays valid. Each check, what it protects, and how to clear it:

#	Gate	Passes when…	If it fails — the fix
1	**Clean tree**	No uncommitted changes in `PythonDataService/` or `references/qc-shadow/`.	Commit or stash your edits. (The running code must match the committed commit the ledger recorded.)
2	**NTP offset**	Host clock is within ±1 s of internet time.	Enable/repair automatic time sync on the host OS. (Accurate time is needed to match fills.)
3	**Unexpected position**	The account holds only an allowed long position in the bot's symbol.	Close any surprise/short/foreign position, or give it its own managed instance.
4	**Run state intact**	`run_ledger.json` exists and is valid.	Don't delete the ledger. If corrupt, re-deploy a fresh run.
5	**No halt flag**	Yesterday did not end in a divergence halt.	Read the `halt.flag` reason, investigate, then deploy a *new* run (a halt can't resume the same run).
6	**Yesterday's receipts**	All of yesterday's trade/reconciliation/QC files still hash to their recorded values.	Don't delete or overwrite a run's output or QC exports between sessions.

Run pre-flight on demand (e.g. offline, skip the network clock check with `--skip-ntp`):

```
python -m app.engine.live.run pre-flight \
  --repo-root /home/inkant/Documents/learn-ai \
  --run-dir live_runs/<run_id>
```

Implementation: `PythonDataService/app/engine/live/pre_flight.py`. A separate, stricter "poison" sentinel exists for *intra-day* fatal divergence (in `halt.py`); it blocks trading until an operator clears it and never resumes on the same run.

10 · Adding a new strategy

There are two ways to add a strategy. For most new ideas, use the **declarative spec** route — it's just a JSON file, no Python required. The **hand-coded** route is for performance-critical or unusual logic and is how the reference strategies were ported.

Route A — A declarative spec (recommended)

A spec is a JSON file that declares indicators and the buy/sell conditions. The engine reads it and runs it through the same backtester and live engine as hand-coded strategies — so a spec you backtest is the same thing you deploy.

Step 1 — Write the spec file

Create `PythonDataService/app/engine/strategy/spec/fixtures/<name>.spec.json`. Skeleton:

```
{
  "schema_version": "1.0",
  "name": "My SPY RSI strategy",
  "symbols": ["SPY"],
  "resolution": { "period_minutes": 15 },
  "indicators": [
    { "id": "rsi14", "kind": "RSI", "period": 14, "source": "close", "ma_type": "wilders" }
  ],
  "entry": {
    "logic": "AND",
    "conditions": [
      { "kind": "IndicatorBetween", "indicator": "rsi14", "lo": 30, "hi": 70, "inclusive":
true }
    ],
    "size": { "kind": "SetHoldings", "fraction": 1.0 },
    "pyramiding": 1
  },
  "exit": {
    "logic": "OR",
    "conditions": [ { "kind": "BarsSinceEntry", "op": ">=", "value": 20 } ]
  },
  "client_id": 11,
  "submit_mode": "live_paper"
}
```

Available indicator kinds include `SMA, EMA, RSI, ADX, MACD, SUPERTREND`. Condition kinds include `IndicatorComparison`, `IndicatorBetween`, `FreshCross` (two indicators crossing), `BarsSinceEntry`, `TimeOfDay`, and PnL/drawdown stops. The full, authoritative schema is the Pydantic model in `app/engine/strategy/spec/schema.py`; you can fetch it live at `GET /api/spec-strategy/schema`.

Step 2 — Backtest it

POST your spec to `/api/spec-strategy/backtest` (or via the strategy-lab UI) with a date range. You get back a trade log and statistics. Iterate the JSON until the behavior is what you want.

Step 3 — Validate & commit

The spec is validated by Pydantic on load (unique indicator IDs, every referenced indicator declared, single symbol, etc.). Once happy, commit the file. It now appears via `GET /api/spec-strategy/fixtures` and in the Deploy form's spec picker.

Step 4 — Anchor & deploy

Run the corresponding QuantConnect backtest, commit the audit copy under `references/qc-shadow/`, then deploy as in §7.

Route B — A hand-coded algorithm

1. Create `app/engine/strategy/algorithms/<name>.py`, subclassing `Strategy` (or `RsiRangeStrategy` for the A/B/C family). Implement `initialize()` (register indicators and a consolidator) and the bar handler (your entry/exit logic).
2. Add a **parity test** proving it matches its reference (golden fixture, explicit tolerance) — this is required for any new math in this repo.
3. Document the port in `docs/references/<name>.md` (source, commit/section, tolerance).
4. Backtest, then anchor & deploy as in §7.

Use the built-in helpers. The repo ships skills that walk these flows: `port-indicator` (bring math in from a reference with proven equivalence) and `add-fastapi-endpoint` (expose new output to the UI). The three reference strategies (A/B/C) under `algorithms/` are worked examples to copy from.

The non-negotiable for any new math. Every ported/added indicator or strategy ships with (a) a golden fixture, (b) a tolerance-pinned test, and (c) a note in `docs/references/`. "Looks right" is not a proof — see `.claude/rules/numerical-rigor.md`.

11 · Troubleshooting & error messages

The UI never makes you parse a stack trace. Every failure maps an HTTP status + the operation into a category and a fix, shown inline next to the action that failed.

Status	Category	Means	Typical fix
400	Validation	An input is bad or missing (spec path, QC path).	Check the spec and QC audit-copy paths exist and are committed.
404	Not found	No run/process for this instance.	Deploy a run first, then start it.
409	Precondition	A gate blocked: dirty tree, run already exists, or already running.	Commit/stash; or stop the existing run before starting another.
503	Infrastructure	The daemon, git, or broker is unavailable.	Start the host daemon / IB Gateway, then retry.

Most common situations

Symptom	Likely cause & fix
"Live engine unavailable" everywhere	The host daemon isn't running. Start it on the host. (It is not started by `restart.sh` / compose.)
Deploy returns 409 "dirty tree"	Uncommitted code in scope. `git commit` / `stash` the listed files, redeploy.
QC audit-copy dropdown is empty	Either no committed files in `references/qc-shadow/`, or (Linux/podman) the container can't reach the loopback-only daemon — bind/start the daemon so it's reachable.
Bot won't trade this morning (BLOCKED)	A pre-flight gate failed. Open the readiness panel; fix the named gate (§9).
"Fleet state blocks new starts"	Account-level state blocks a new start. Deploy-only now (untick start), or clear the account state, then start from the console.
Halt flag set; can't resume	Yesterday diverged. Read the halt reason; deploy a fresh run (same run can't resume after a halt).

> **Where to look on disk.** A run's evidence lives in `live_runs/<run_id>/`: `run_ledger.json` (identity), `decisions.parquet` (one row per bar), `trades.parquet` (completed trades), `host_daemon.log` (process output), and `reconcile/` (daily reports + hash sidecars).

12 · Quick-reference cheat sheet

Daily "clean deploy" routine

1. `git status` → commit/stash anything dirty.
2. Host: start IB Gateway (paper) + the daemon.
3. Confirm QC backtest ID + committed audit copy.
4. Open `/broker/deploy`; fill all fields.
5. Fleet amber? untick "start now," deploy-only.

Key commands

```
run init-ledger ...   # create a run
run start --run-dir   # launch engine
run pre-flight ...    # morning checks
run pause|resume|stop --strategy-instance-id
```

6. Deploy → start from console when clear.

The two checklists

- **A · Deploy** — engine up, broker, fields, fleet clear; then clean tree + spec match.
- **B · Pre-flight** — clean tree, NTP, no surprise position, ledger intact, no halt flag, yesterday's receipts.

Readiness verdicts

- READY trade allowed
- BLOCKED hard gate failed
- DEGRADED soft warning, can trade
- UNKNOWN not running

```
ID
run emergency-flatten # close all (panic)
```

Key locations

- Console: `/broker/instances`
- Deploy: `/broker/deploy`
- Specs: `app/engine/strategy/spec/fixtures/`
- Algorithms: `app/engine/strategy/algorithms/`
- QC anchors: `references/qc-shadow/`
- Run output: `live_runs/<run_id>/`
- ADRs: `docs/architecture/adrs/`

Status codes

- **400** bad input · **404** not found
- **409** blocked (dirty tree / exists)
- **503** daemon/git/broker down

Remember

- Paper account only (`DU…`). Keep mode = paper.
- No QC anchor → no deploy.
- Daemon + Gateway are host programs, not containers.
- Instance = the bot you manage; run = one switch-on.

> **If you remember one thing:** commit your code, bring up the host (Gateway + daemon), and have your QC anchor committed — *before* you open the Deploy form. That keeps both checklists green and your deploys clean.

This manual describes the live-trading subsystem of the **learn-ai** research platform as of 2026-06-01. It is a research/education tool; nothing here is financial advice, and real-money trading requires separately validated infrastructure. For the authoritative "why" behind any rule, see the ADRs in `docs/architecture/adrs/` and the rules in `.claude/rules/`.